

```

        X = x;
    }
    /// <summary>
    /// Implicitly converts from ints to MyTypes.
    /// </summary>
    public static implicit operator MyType(int x)
    {
        return new MyType(x);
    }
    /// <summary>
    /// Explicitly converts from MyTypes to ints.
    /// </summary>
    public static explicit operator int(MyType x)
    {
        return x.X;
    }
}
public class MyTypeTest
{
    public static void Test()
    {
        int x = 10;
        MyType m = new MyType(5);

        MyType n = x;
        int y = (int) m;
    }
}

```

3.4 Partial classes and extension methods

Finally, the most recent C# features have given programmers more flexibility in extending classes. The first of these features is partial classes which allow you to split the definition of a class over multiple files.

One use of partial classes is to solve the class design issues. We can get exhaustiveness checking of a class hierarchy with the code locality of like-behavior by using partial classes to break up classes into different files.

```

// in Entity.cs
abstract partial class Entity { / ... / }

```